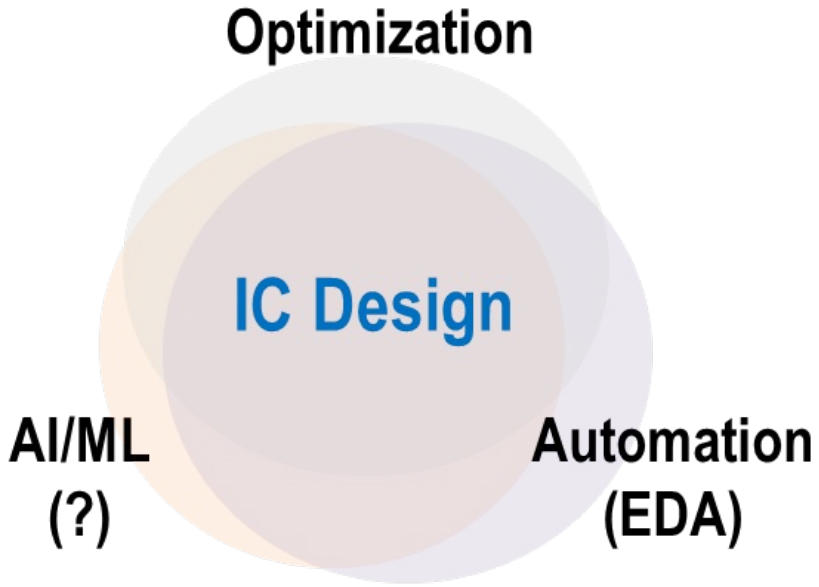
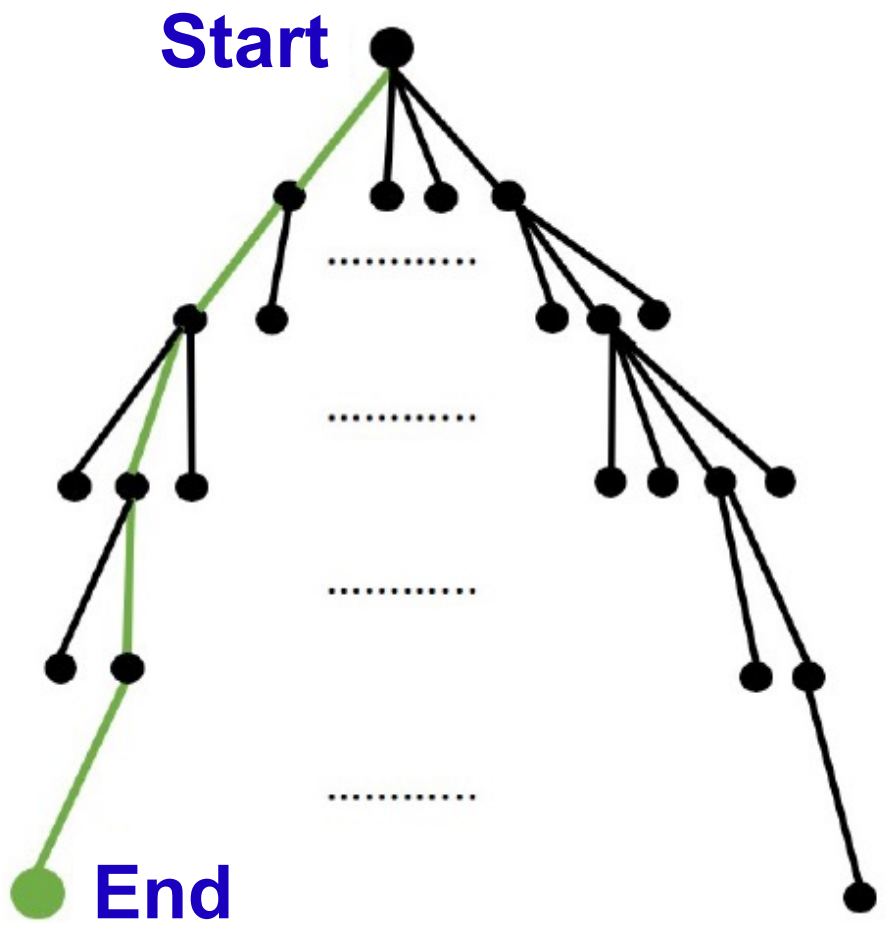
On AI and EDA /
On OpenROAD in the Era of Agents

ECE 260C, Spring 2026
Andrew B. Kahng



Design Optimization Lives in a Box



- **Start** to **End**: expensive!
  - O(year) for product
  - O(weeks) for SP&R and Opt
- Goal: best possible **End**
- Constraint: stay in "Box"

  {compute}
  X {licenses}
  X {people}
  X {weeks}

Designers always need more leverage!

"Machine Learning in EDA": Why

A. You need models to have predictions

B. You need predictions to leverage in exploration

C. What you can't predict, you guardband

D. What you don't explore, you leave on the table

E. C and D are bad for product quality and schedule

"Moore's Law slowdown" → in an Era of Optimization

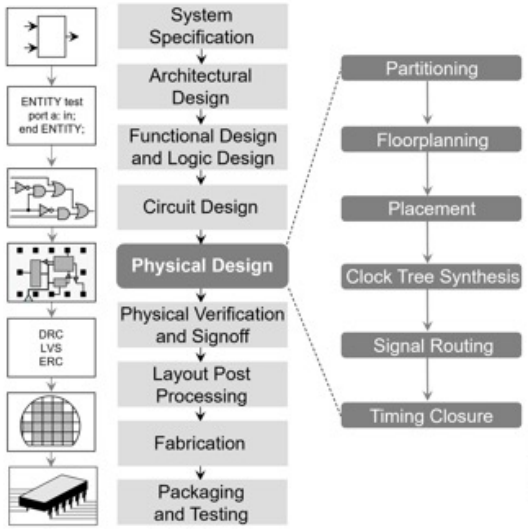
"Machine Learning in EDA": What

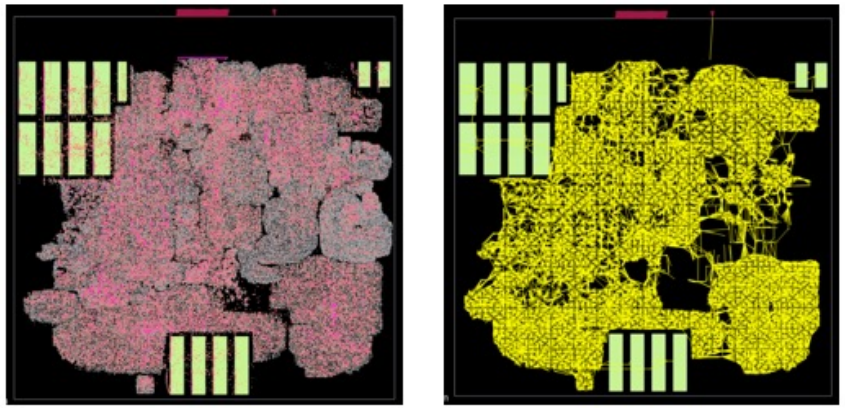
- **Predict**
  - Will RouteOpt finish with clean signoff, <1000 DRVs by tomorrow night?
- **Classify**
  - Out of these 50 floorplans + budgets, which 3 should go into trial SP&R?
- **Estimate**
  - How many hold buffers will tool eventually add into this post-CTS layout?
- **Guide / advise**
  - What P&R tool setup/script will obtain the best QOR within next 36 hours?

- More broadly: answer any question that is difficult for humans
  - Google Brain, 2020: "super-human macro placement" on arXiv
  - See: Rich Sutton, "The Bitter Lesson"
- More trivially: automate timing / DRC fix; help choose flow script; …

What's Different (and Difficult) for AI/ML?

1. Changing abstractions, formats, and the design itself





LEF, DEF, .LIB, SVRF, UPF, CPM, APL, SPEF, SDC, GDS, .V, VHDL, .SP, ITF, TLU+, NXTGRD, PFM, QRCtech, SDF, CCS, LVF…

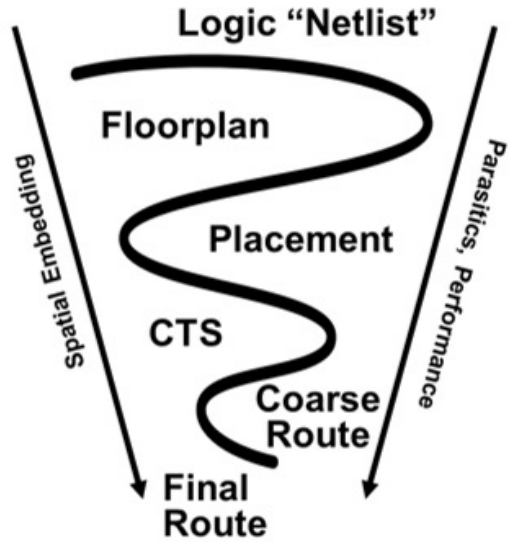
2. Long chains of distinct, intractable discrete optimizations

- "Practical optimization" = metaheuristics on top of metaheuristics
  - *Scale, multimodality, dynamism, diversity*
    → 1000s of hidden commands and options in a commercial placer !
- Objectives are **ad hoc**
- Trajectories are **chaotic**
- Outcomes have **distributions**

3. Loops are expensive

(often, fatally so)

- Design process must "converge" both spatial embedding **and** performance

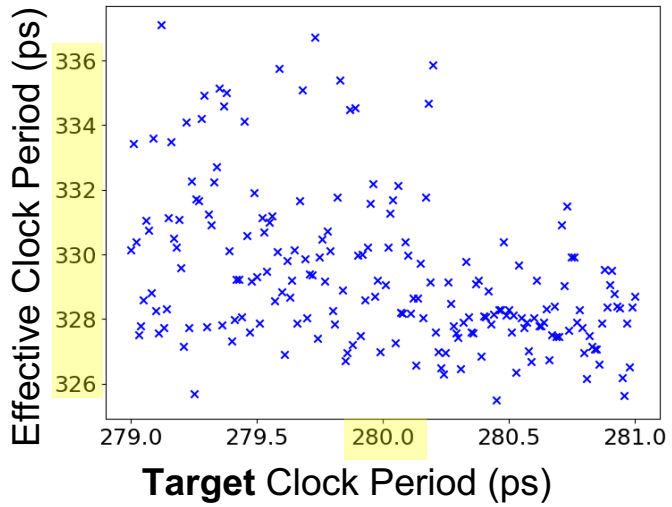


How?

Chained Chaotic Heuristics vs. Surrogates, Predictors

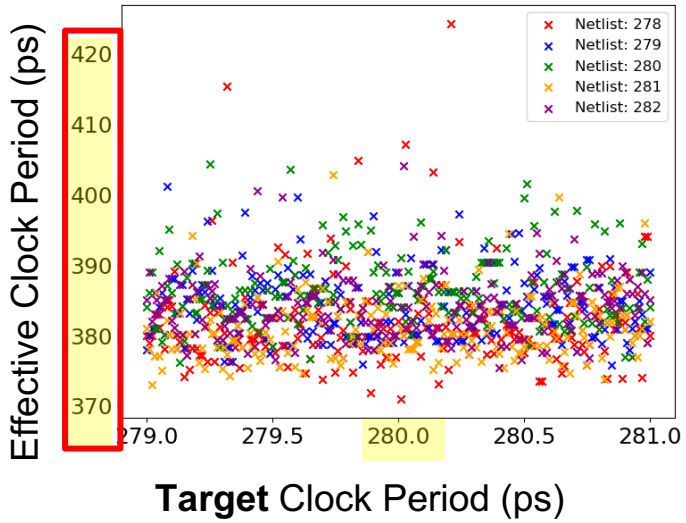
Logic Synth

3.4% **Variation of Effective Clock Period**

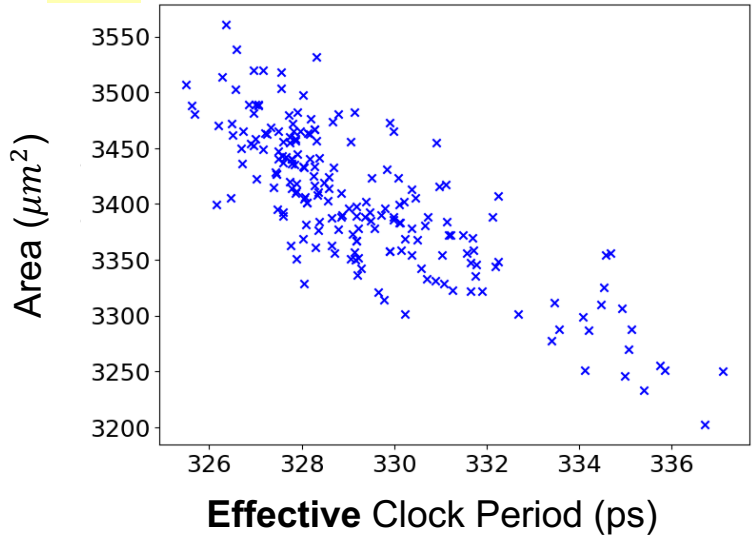


Place & Route

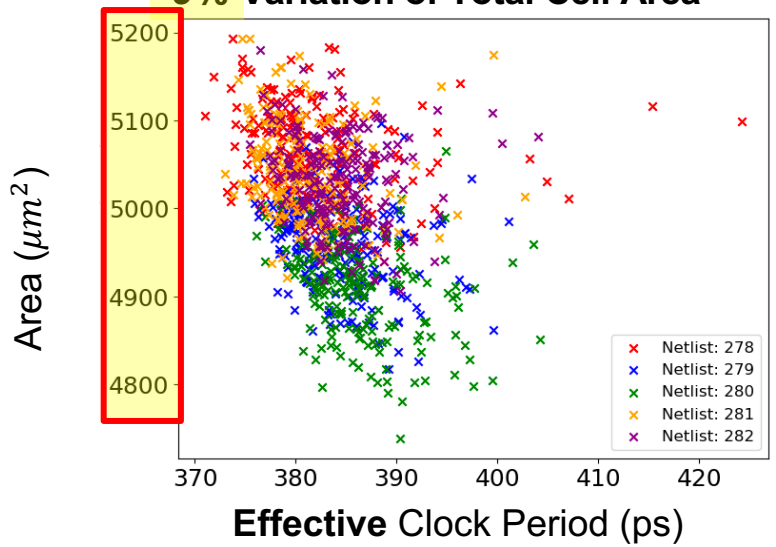
15% **Variation of Effective Clock Period**



11% **Variation of Total Cell Area**



9% **Variation of Total Cell Area**



Variation of metrics: $100 \times (\frac{max}{min} - 1)$

Implications for Optimization → Implications for AI/ML

- Predictions today are **Constructive**
  - Quick-and-dirty, under the hood

Catechism:

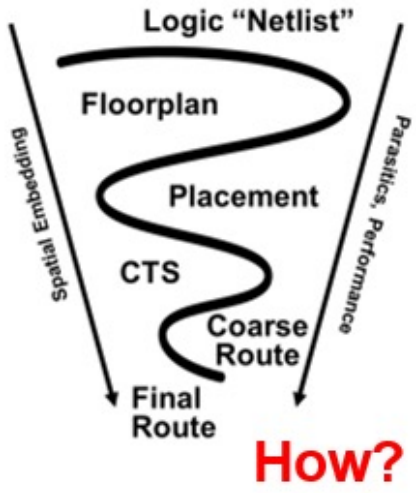
A. You need **models** to have predictions

B. You need predictions to **leverage** in exploration

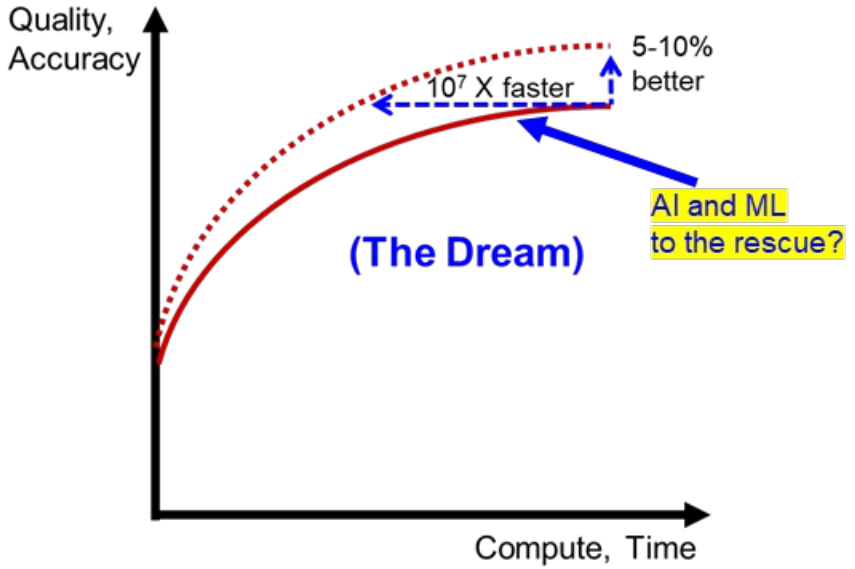
C. What you can't predict, you guardband

D. What you don't explore, you leave on the table

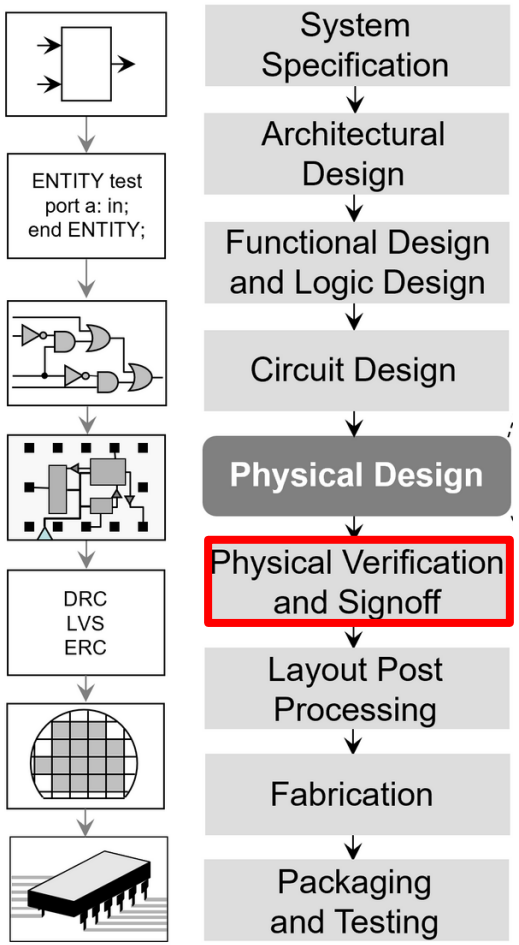
- Optimizations today are **Iterative**
  - "Construct by Correction"



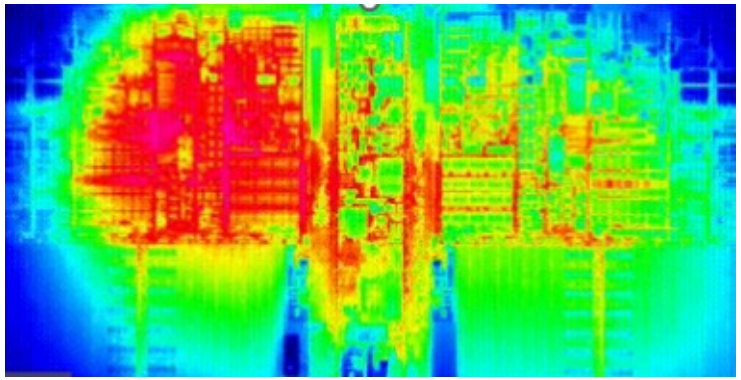
- **Need Fast Simulations** to guide Optimization
  - "Construct by Corrections …
    … that are Correct by Construction"

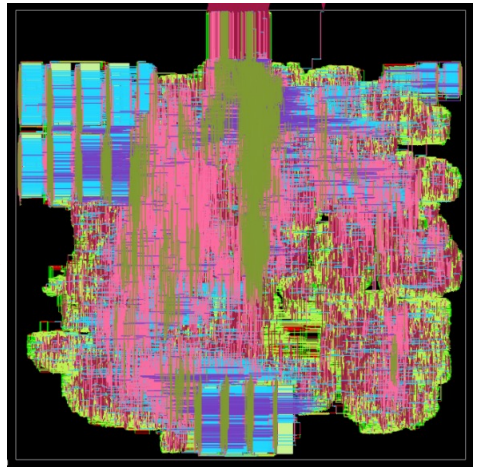


WARNING: Signoff in Design: Optimize Max, not Sum!

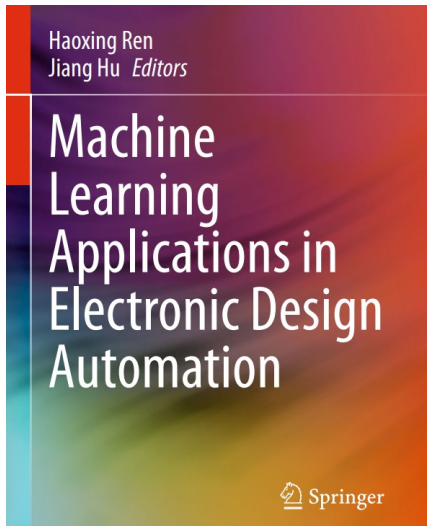


- **Signoff** is a defined business interface
  - Establishes "Who pays for the scrap?"
- Golden, foundry-qualified tools perform signoff analyses and simulations
  - Typically with very long batch runtimes
- Many design steps must optimize "max"
  - Max timing path delay determines max frequency
  - Max wiring congestion determines routing feasibility
- Inverse problems galore
  - Worst-case stimuli → e.g., "rogue wave in power grid"
  - "Whack-a-mole", "ping-pong" are in the lexicon of design → can new AI foundation models help?

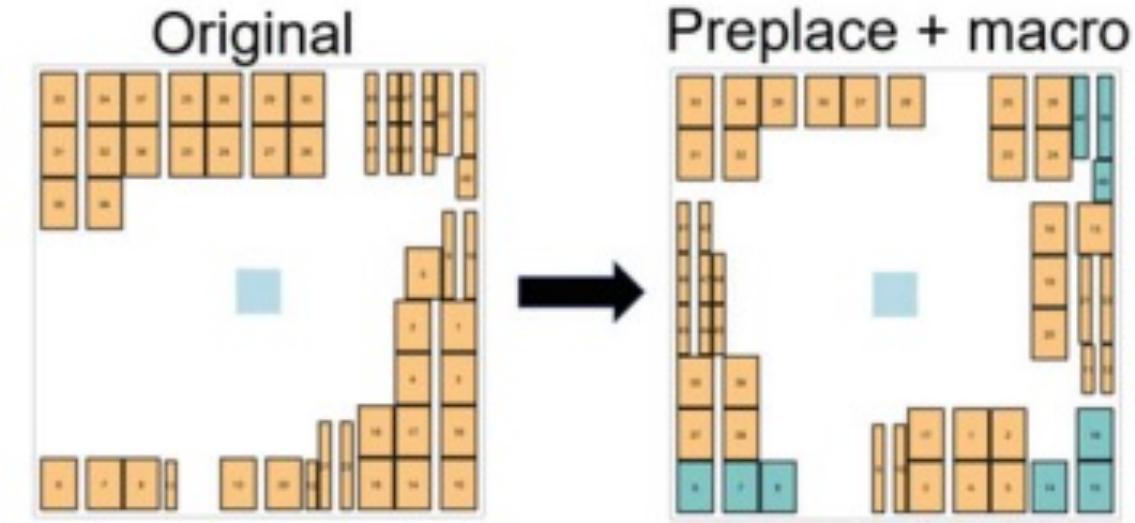




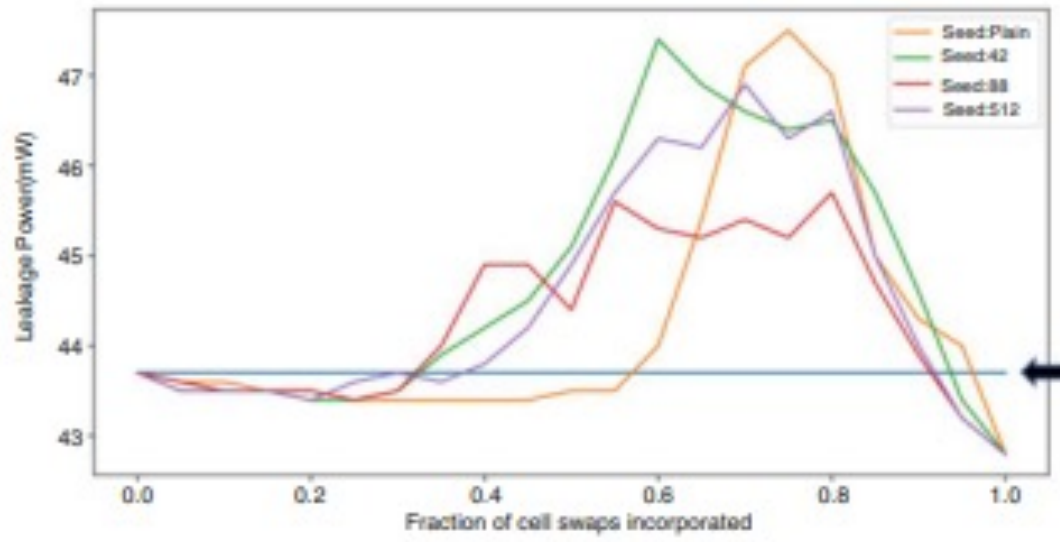
WARNING: "Be Careful What You Ask For" (from AI/ML)



- **"ML for Design QoR Prediction"** (Chapter 1 in this book)
- Acting on a prediction can change what is being predicted
  - **Upper figure**: knowing + fixing the blue macro placements changes the outcome
  - **Lower figure**: knowing + making ~70% of leakage ECO swaps consistently worsens outcome
- We want **actionable** and **useful** ML predictions !



P&R_1



ECO_2

Original

WARNING: "What You Ask For Is Not What You Get"

- From this paper in IEEE Design & Test MLCAD issue (February 2023)
- Can we learn the **Ask → Get** mapping?
- Red dots = flow failures → must "PPA prediction" be probabilistic?
  - "for a 50+% chance, make sure to press the button 100+ times" …

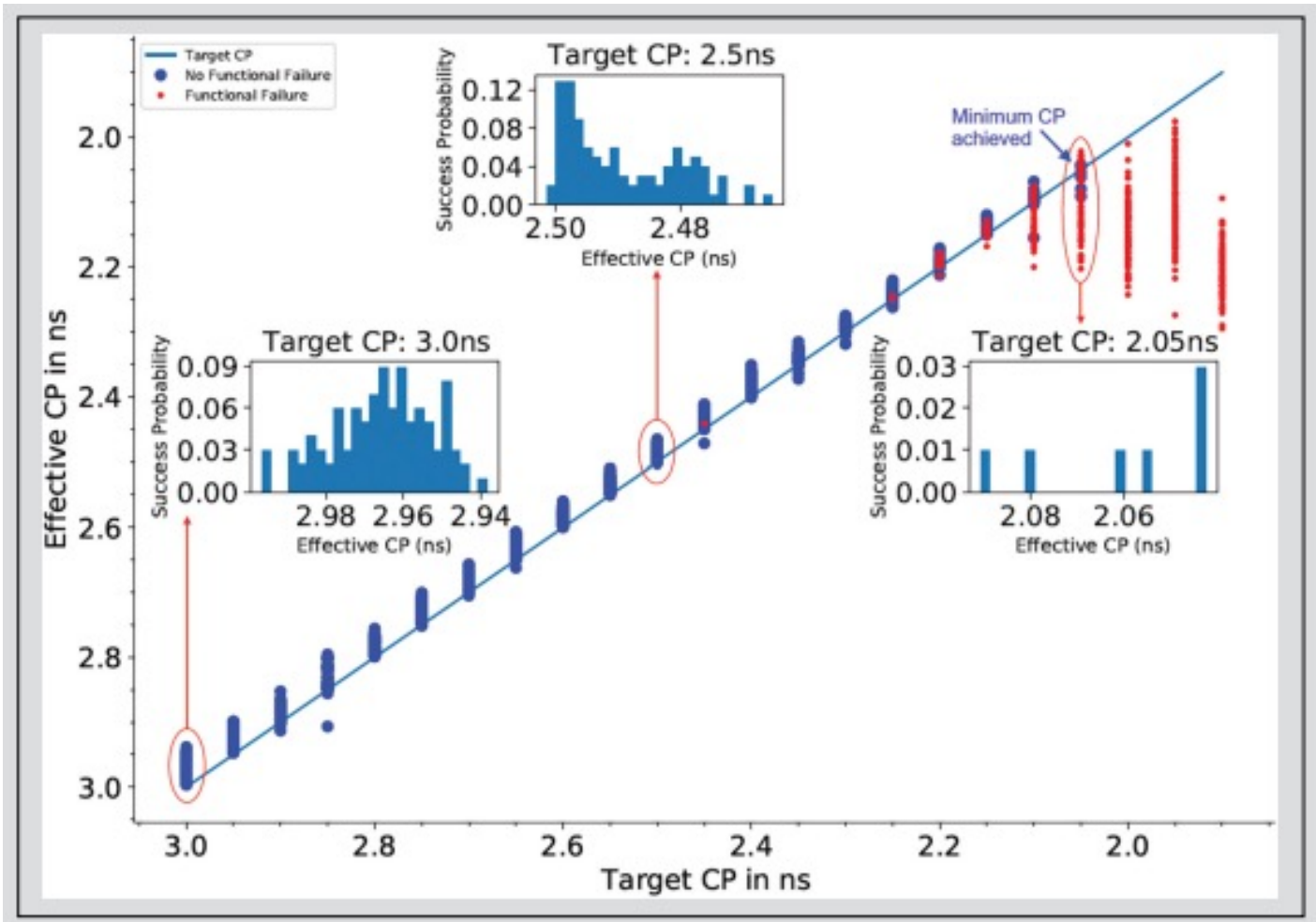
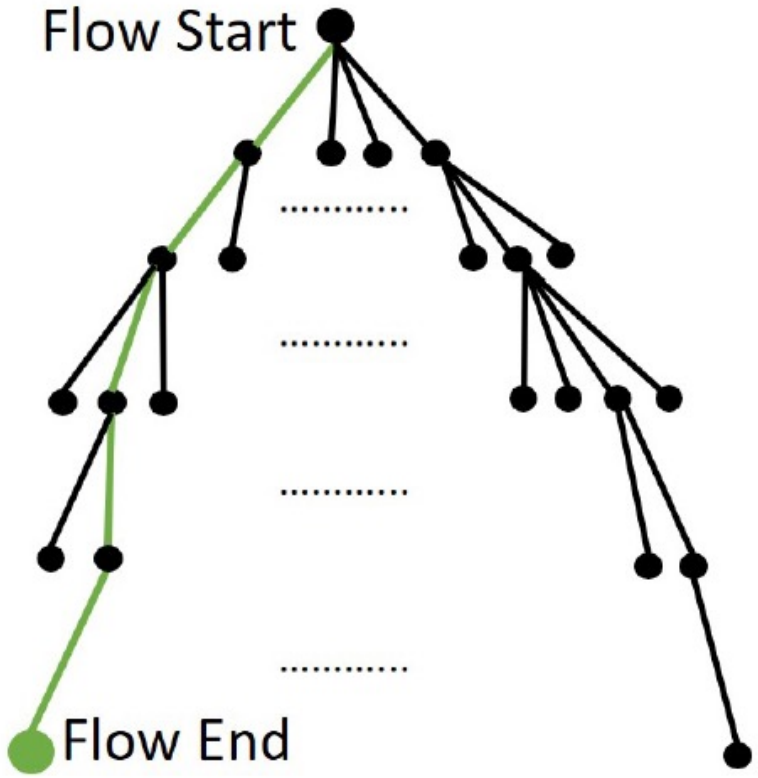


Figure 5. Mapping between desired (target) CP and actual (effective) CP achieved by a commercial place-and-route tool. At each *x*-axis value, results are obtained for 101 target CP values within a 1 ps range of the given target CP. The highest blue data point in the plot corresponds to the minimum CP achieved, that is, the maximum-frequency result.

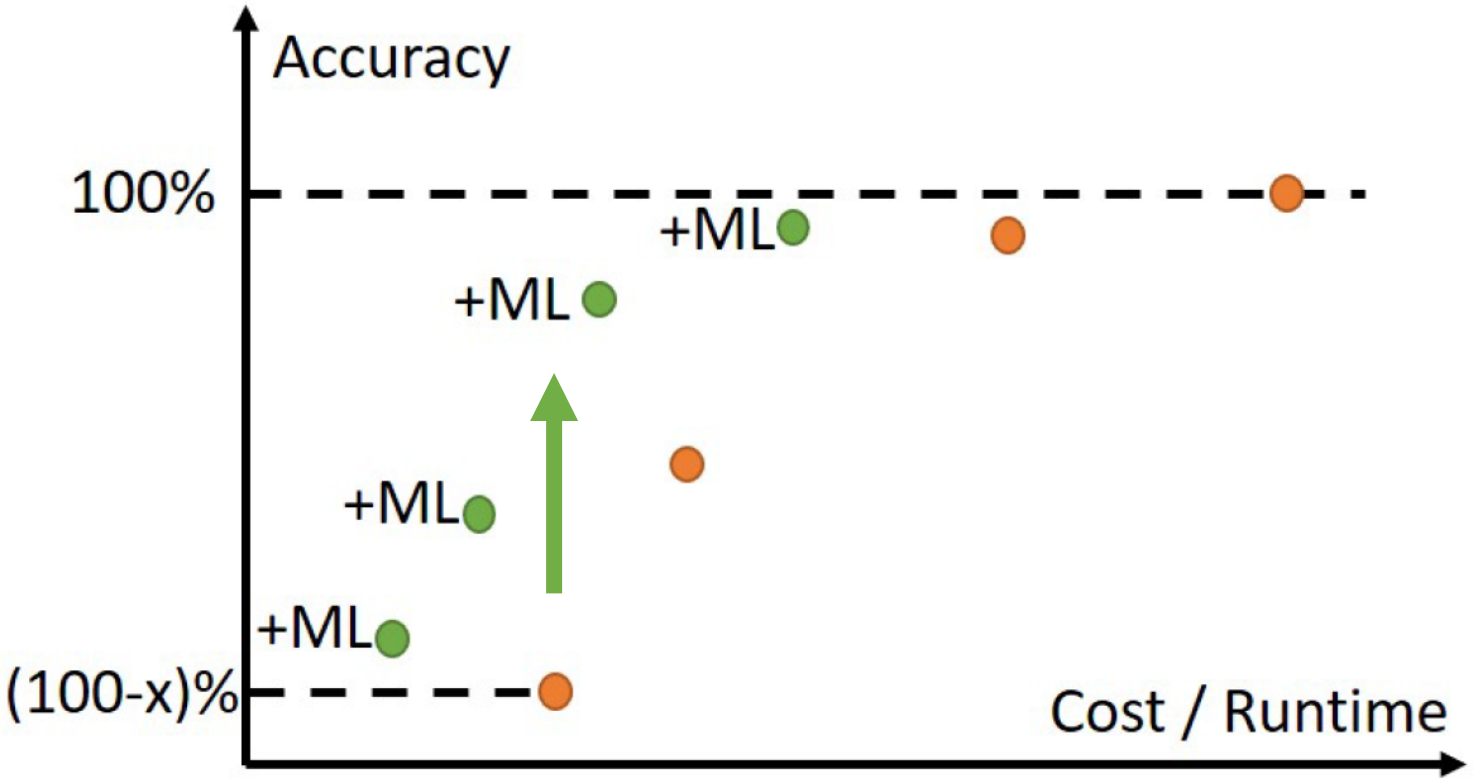
4-Stage "Roadmap" of ML in EDA

1. Mechanization and Automation
2. Orchestration of Search and Optimization
3. Pruning via Predictors and Models
4. From Reinforcement Learning through Intelligence



Huge space of tool, command, option trajectories through design flow

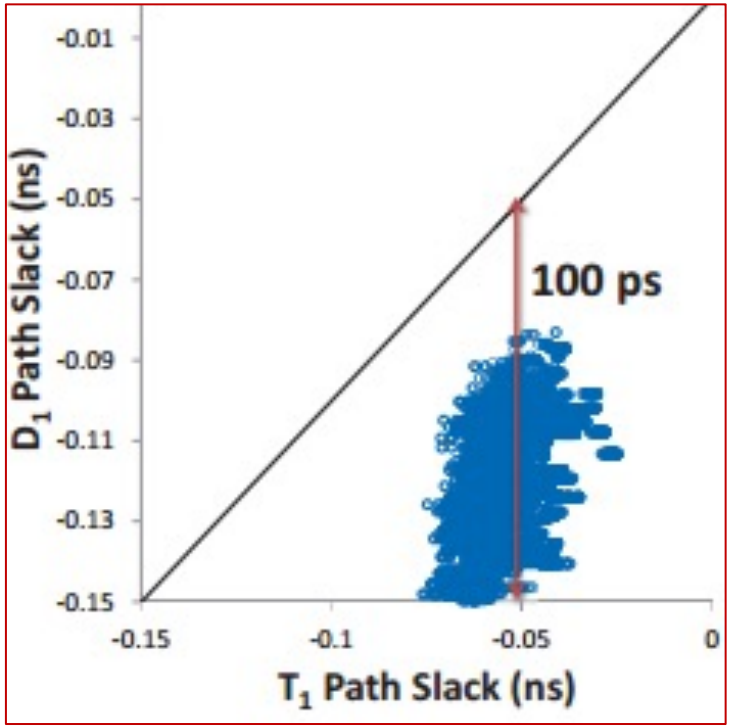
No-Brainer: Shift Accuracy-Cost Tradeoffs

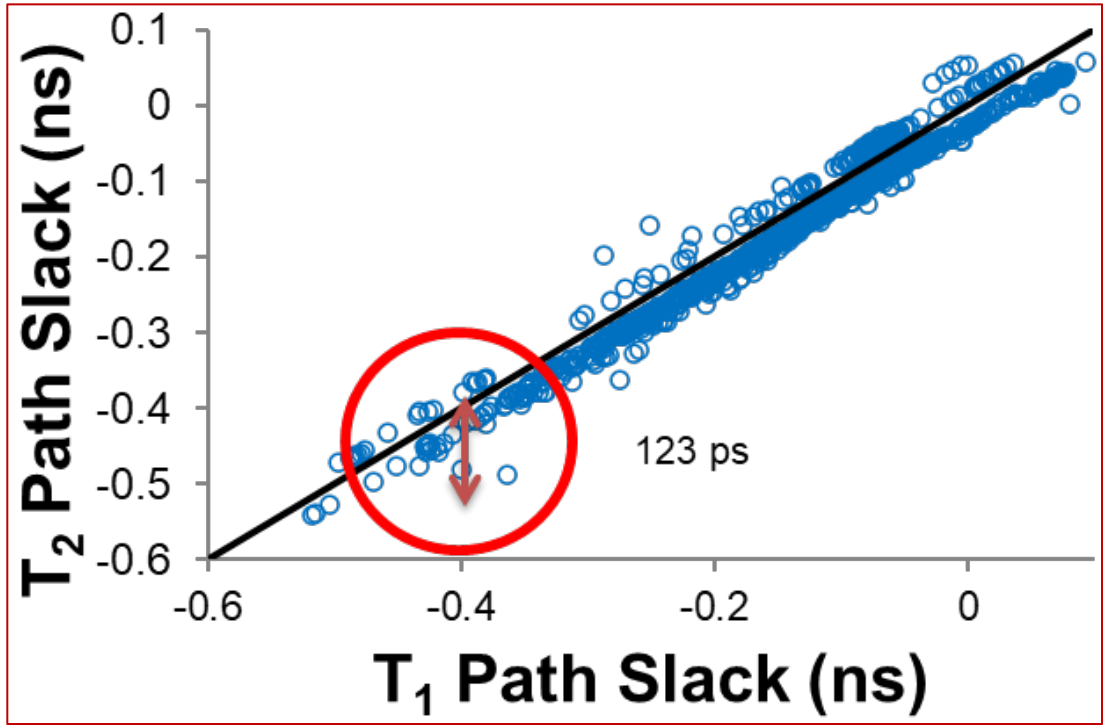


Tagline: "It's Just Physics!"

ML to Fix Timing Miscorrelation

UCSD, DATE-2014



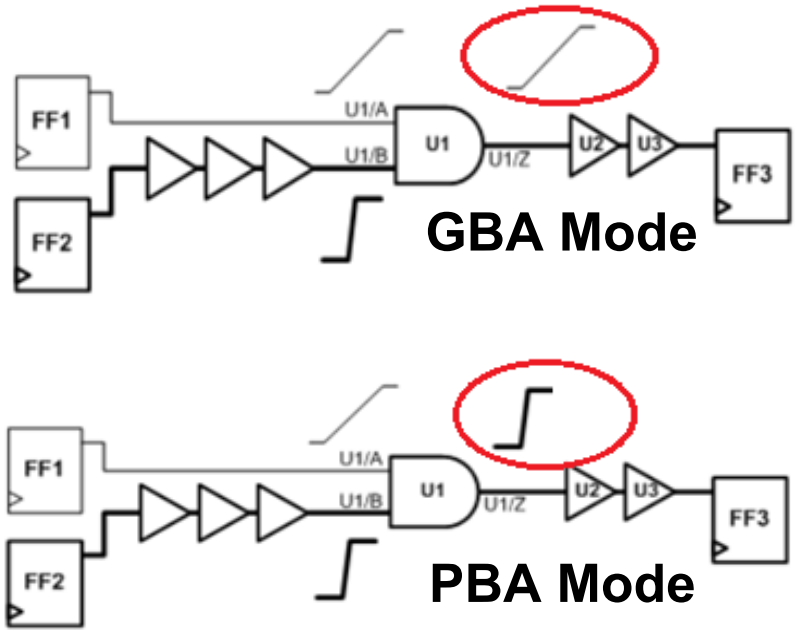


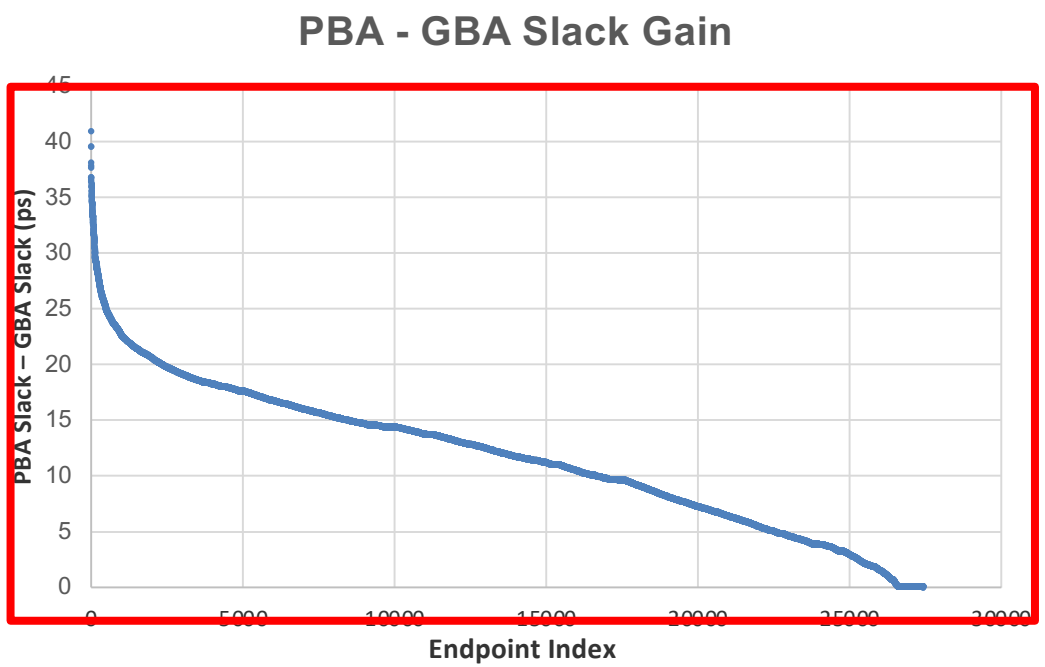
D = non-signoff timer (e.g., in P&R tool)
T = "golden" timer (e.g., signoff-qualified)

- **Can you explain the slack miscorrelation?**
- **What is the impact of the miscorrelation?**

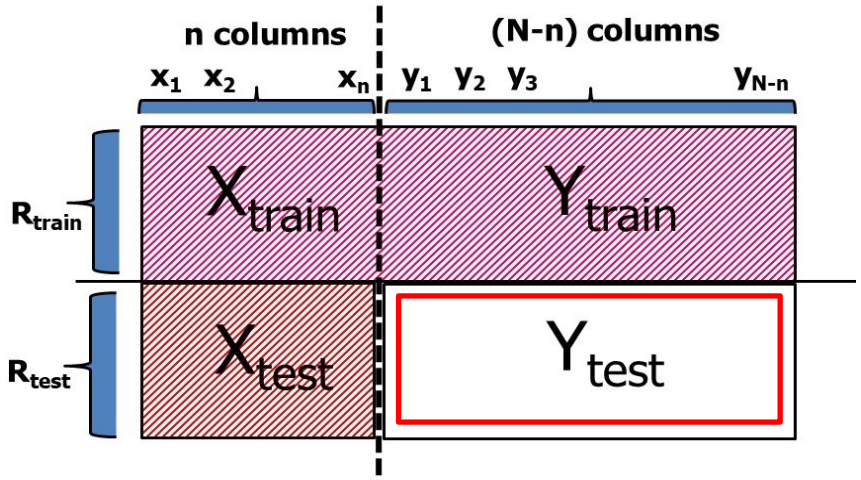
Predicting PBA from GBA

- PBA (Path-Based Analysis) is less pessimistic but more expensive than GBA (Graph-Based Analysis)
- **ML to predict PBA timing from GBA timing**
  → Better and faster outcomes from P&R, Opt

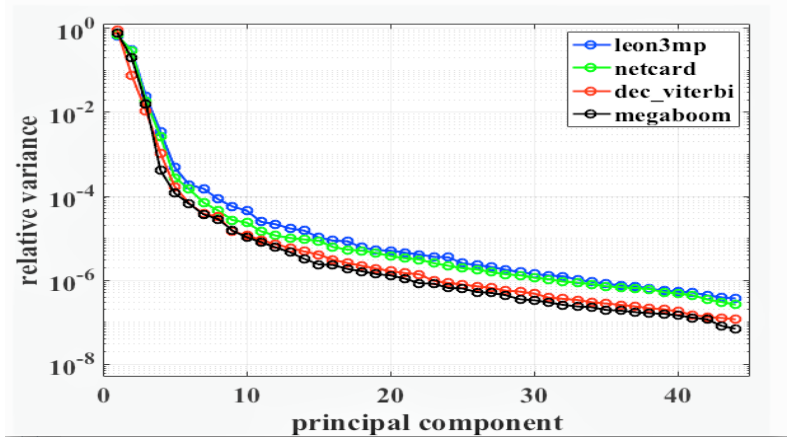




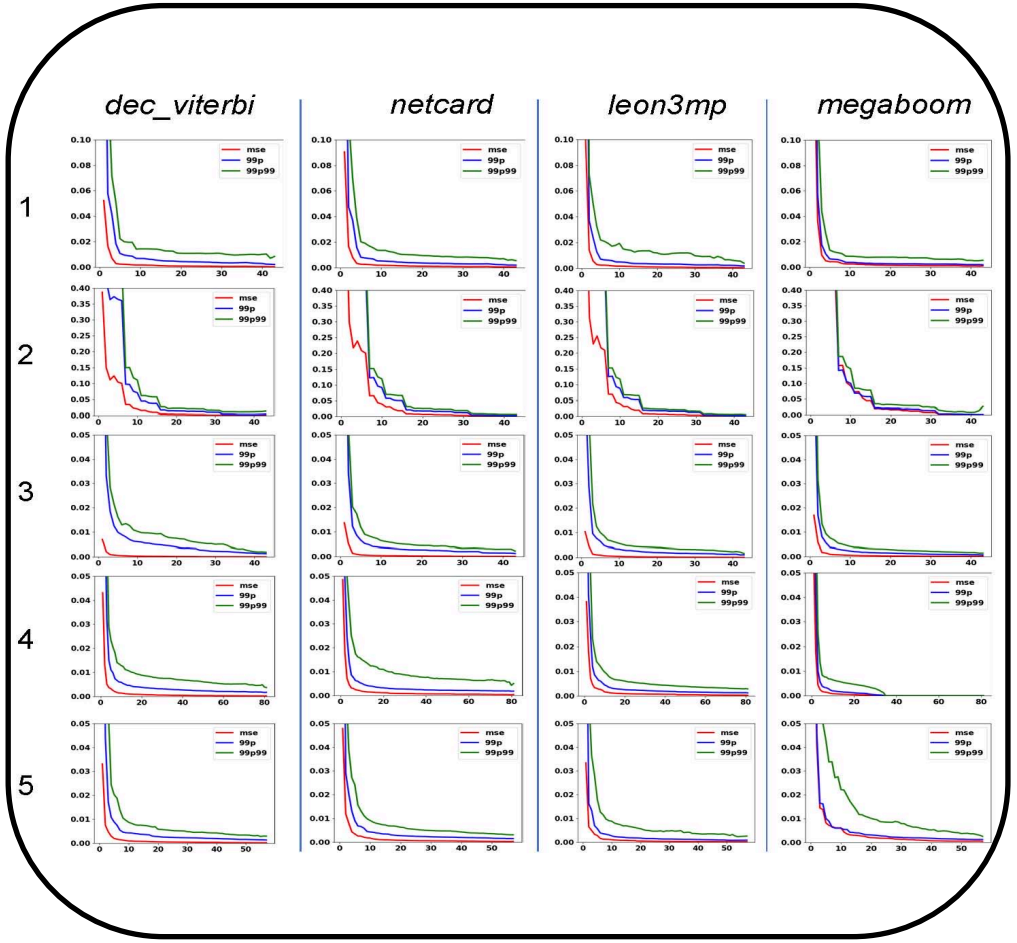
Timing at "Unobserved Corners"



Predicting missing delay values
= *matrix completion* problem



PCA: low-dimensional modeling task

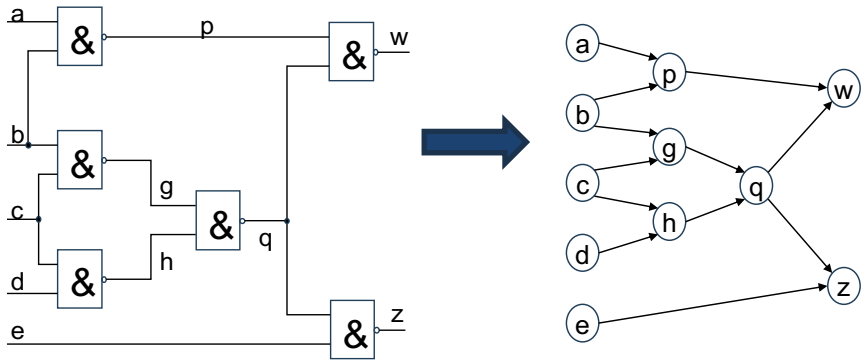


STA at few *known corners* → predict timing at all *unknown corners*

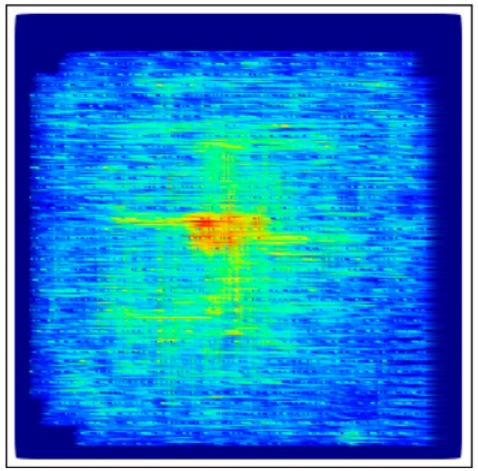
"It's Just Physics!"

Call-Out: Semiconductor Design Data

- **Many types**
  - Formal specs
  - HDLs
  - Graphs
  - Hierarchies
  - Tabular data
  - Images




```
// Memory Write Block
// Write Operation : When we_0 = 1, cs_0 = 1
always @ (address_0 or cs_0 or we_0 or data_0
or address_1 or cs_1 or we_1 or data_1)
begin : MEM_WRITE
  if ( cs_0 && we_0 ) begin
     mem[address_0] <= data_0;
  end else if  (cs_1 && we_1) begin
     mem[address_1] <= data_1;
  end
end
```



- **IC data characteristics**
  - Constantly changing
    - technologies, designs, tools, …
  - Non-standard forms (even, “Tower of Babel”)
  - No massive redundancy
  - Different shapes and scales
  - No “Zipf’s Law”

- **Is unsupervised learning even possible?**
- **Who owns, and who can use, what data?**

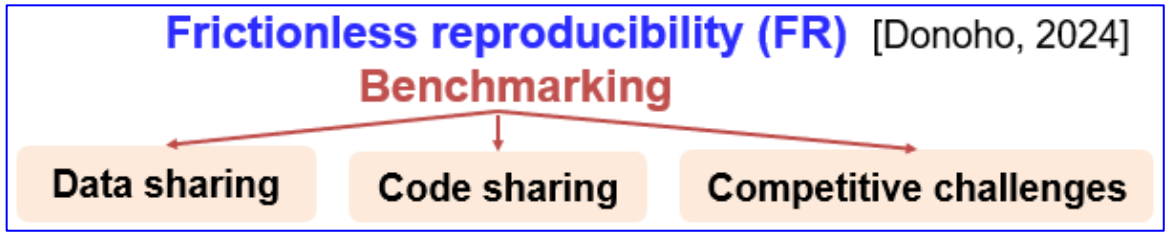
Thanks: Igor Markov, Synopsys

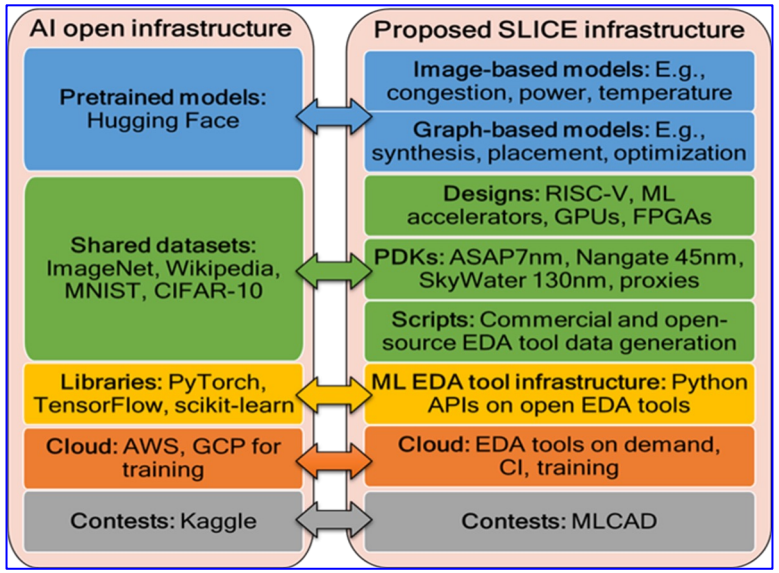
Call-Out: Semiconductor Design Data

- **Proprietary**
  - Designs
  - Technologies
  - Design methods
  - EDA tools
- **Expensive**
  - E.g., design flows take weeks to run
- **Closer to physics → harder to access**
  - Materials
  - Equipment
  - Process, Devices
  - …

Semiconductor Design Data: Well-Lamented Gaps

- **Proprietary**
  - Designs
  - Technologies
  - Design methods
  - EDA tools
- **Expensive**
  - E.g., design flows take weeks to run
- **Closer to physics → harder to access**
  - Materials
  - Equipment
  - Process, Devices
  - …



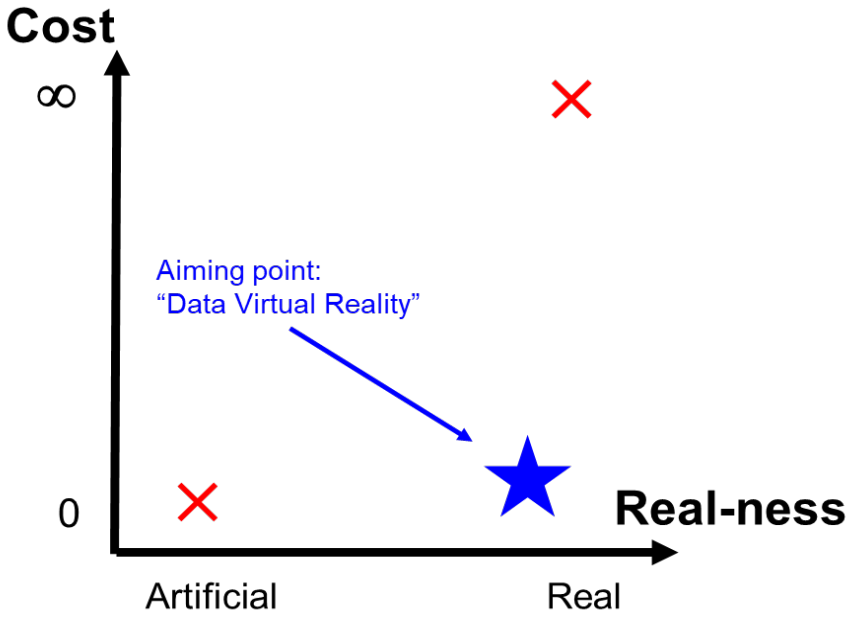


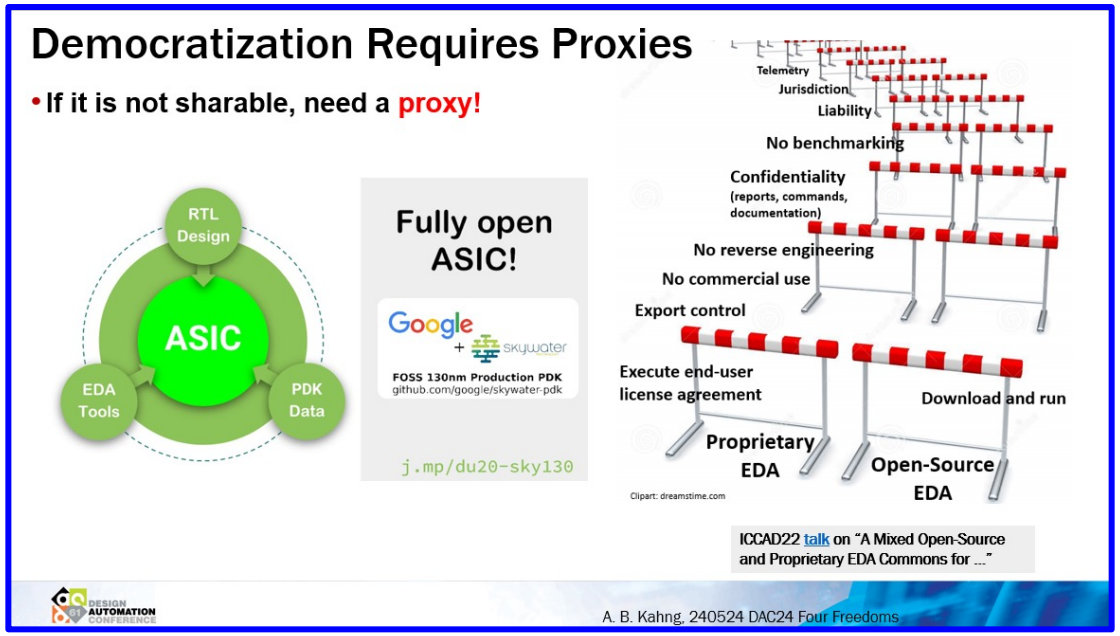
https://slice-ml-eda.github.io/

A Call to Action: Must Develop "Proxies"

"if it can't be shared, need a proxy !"

- Proprietary PDK (Process Design Kit) data
- Commercial EDA End-User License Agreements
  - **No benchmarking (!)**
  - **Copyrighted command language → Tower of Babel**
    - report_timing, report_checks, report_timing_analysis, check_timing, …
  - **Copyrighted report formats → more Tower of Babel**
    - ^ and v vs. r and f , *** vs. === , …
  - Can't share/upload ML data or models!
- **Foundation Models** will require **Data**, which will require **Proxies** !
  - Tech files, device models, "safe names", design enablements / tool setups, sharable results and metrics, …
    ("journey of a thousand miles …")





OpenROAD and the Era of Agents

Andrew B. Kahng

University of California, San Diego

abk@ucsd.edu

Outline

- **The agentic era in EDA**
- **Case studies in OpenROAD**
- **Emerging challenges for agentic open-source EDA**

The agentic lever *(LLMs → Agents → R&D Acceleration)*

- Scaling is **design-based**
- Tools are **bespoke**
- Ecosystem is **consolidated**
- **Optimization imperative**
  = do better and faster, with less

LLMs in EDA (2026 ++)

- Multi-hours/multi-days reasoning
- Comprehend large codebases
- Invoke tool commands
- Self iteration with feedback

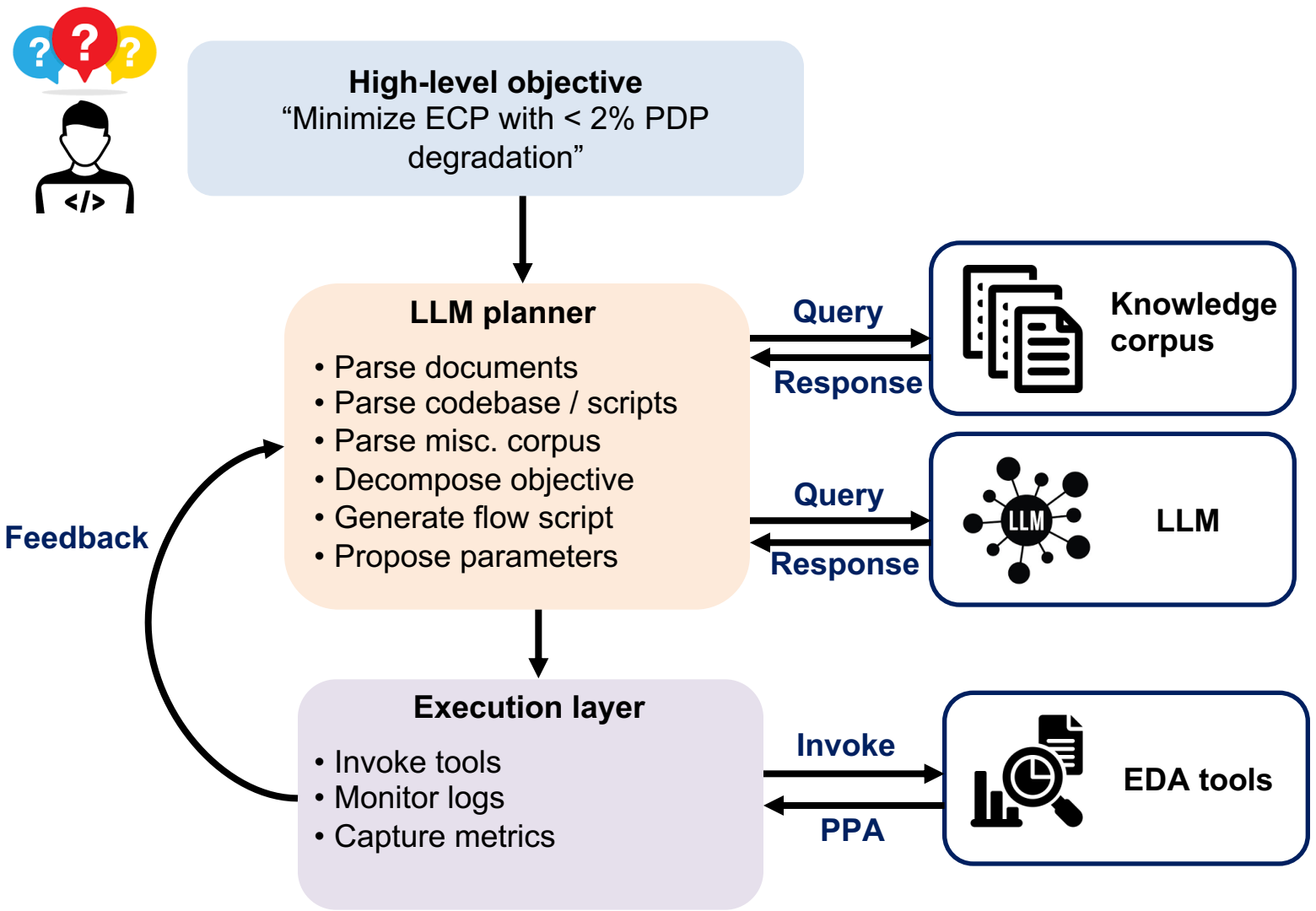
IC design optimization (PPA, schedule, cost) → **LLMs + tool use** + **EDA = tools** (synthesis, P&R, STA, etc.) → **(Human +) Agentic R&D loop** (read logs, tweak params, edit code, check/eval + iterate)

From "EDA tools" → to "EDA R&D collaborators"

From assistance to autonomy: two levels

- **Fact:** Agents (in EDA) will operate across differing levels of **abstraction**
  - **Abstraction** determines **search space**, **feedback signal**, and **risk**
  - ***Open-source EDA is the perfect (and our only) testbed !!!***
- **Two levels (~patterns)**
  - **Code-level agents** modify algorithmic structure (source code)
    ← "Improve the global swap operator in the dpl module of OpenROAD"
  - **Flow-level agents** modify control variables (tool flow, parameters, objectives)
    ← "Minimize ECP with < 2% area/power/PDP degradation by tuning ORFS configs"
- *Is the agent reasoning about **heuristics** (code and operators) or about **policies** (parameters and sequencing)?* *(Inevitably, agents will reason about both!)*

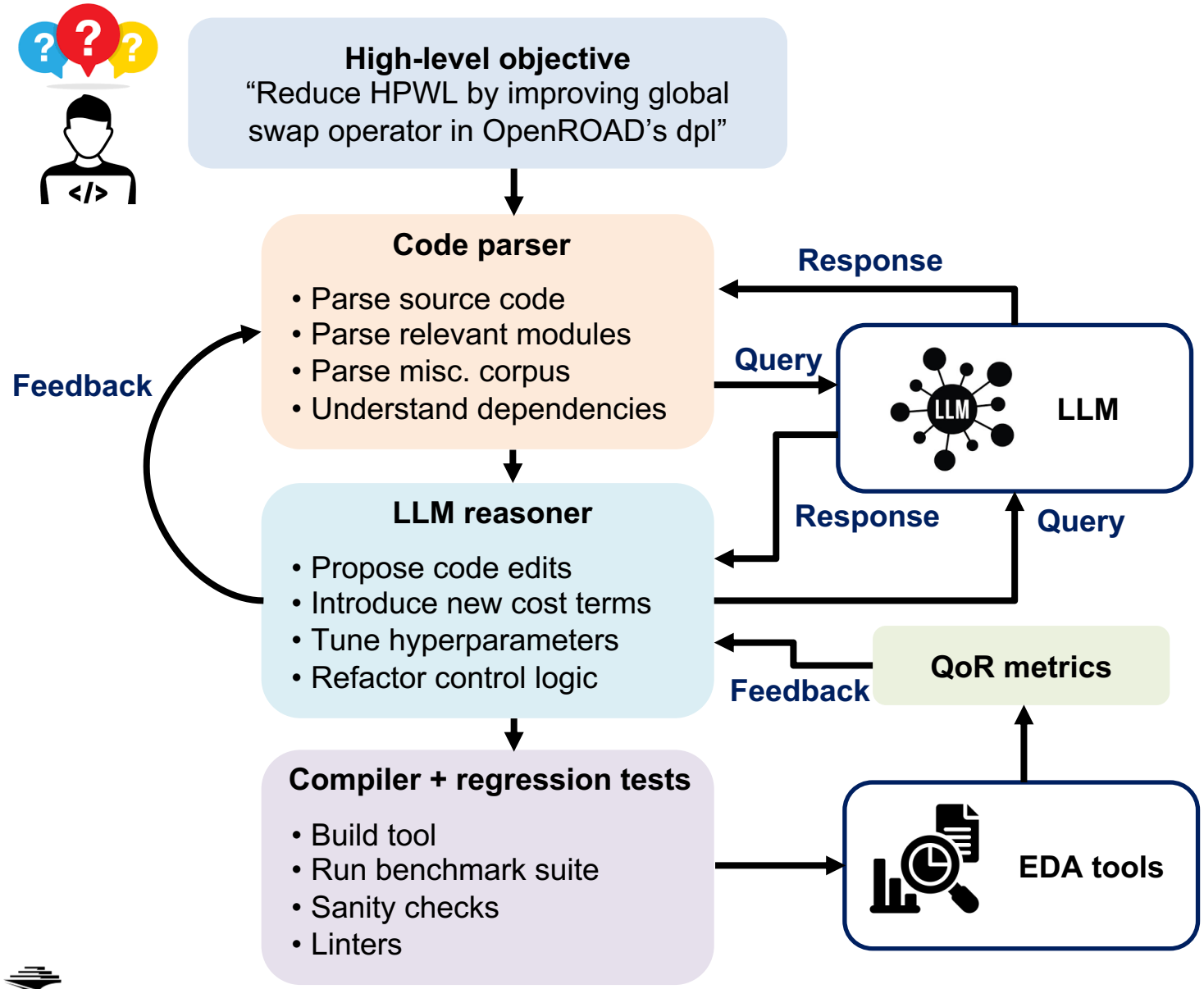
Flow-level agents – architecture



- **ChipNeMo (NVIDIA) (2023)**
  - Pretraining on expert-engineer scripts
  - Combines ICL with document retrieval
  - Generates high-quality Tcl scripts for timing analysis, etc.
- **ChatEDA (CUHK) (2024)**
  - Translates flow automation to an agentic task
  - Rich training corpora, CoT prompting, usage of multi-agents
- **ORAssistant (Precision Innovation) (2024)**
  - Automated script generation and chatbot for OpenROAD ecosystem
- **ORFS-agent (UCSD) (2025)**
  - LLM-driven iterative optimization agent for parameter tuning in the OpenROAD-flow-scripts repo.

Treat EDA tools as black-boxes; LLM decomposes objective into subtasks; each subtask is translated to scripts

Code-level agents – architecture

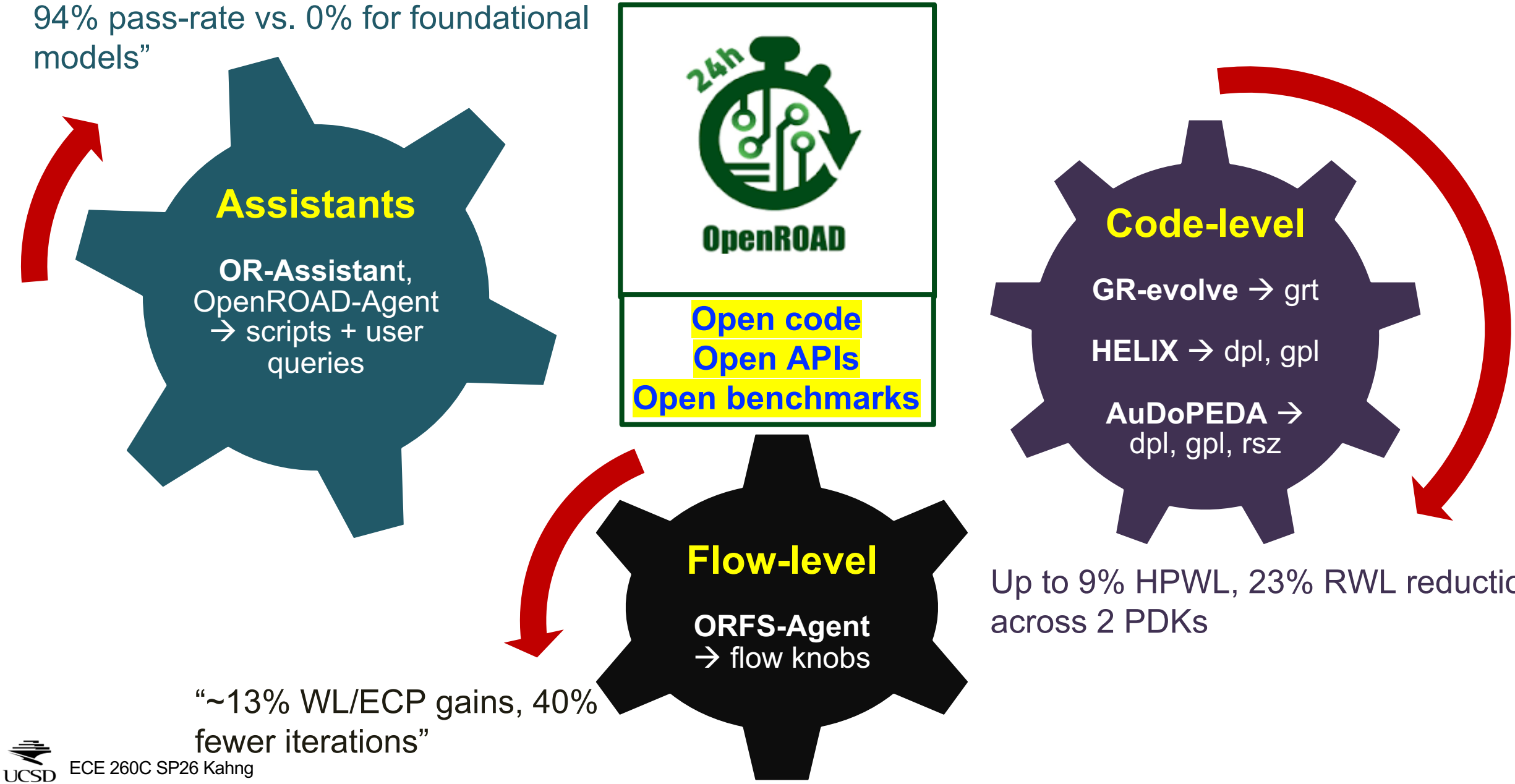


- **EvoPlace (CUHK) (2025)**
  - Python code edits inside DREAMPlace framework
- **SATLUTION (NVIDIA) (2025)**
  - C++ code diffs targeting Boolean Satisfiability problems
  - Directly evolves solver repositories
- **-EVO (UCSD) (2025)**
  - C++ code diffs targeting placement algorithms in OpenROAD
  - Directly evolves placement repositories
- **AuDoPEDA (UCSD) (2026)**
  - Proposes research directions + expands them into executable code diffs
- **VLM-MPL (UCSD) (2026)**
  - Vision language models-based macro placer

Modify internal heuristics and algorithms; LLM reads, proposes and edits code diffs

OpenROAD as a foundation for agentic EDA research

94% pass-rate vs. 0% for foundational models”



Up to 9% HPWL, 23% RWL reduction across 2 PDKs

“~13% WL/ECP gains, 40% fewer iterations”

Three frontiers of recent progress in OpenROAD

Flow tuners

- Treat EDA tools as **black-boxes**; **tune** parameters and orchestrate flows
- **ORFS-Agent** (UCSD)
  OpenROAD-Agent (ASU)
  ORAssistant (PII)
- **Algorithms unchanged**; agents learn the policy

"~40% fewer iterations than autotuners"

Repo-scale coding

- Agents can **read, edit, compile** and **benchmark** across **multi-language codebases**
- **AuDoPEDA** (UCSD)
- **Algorithms change**; formal verification; closed-loop guardrails

"~19% power gains"

Bespoke, design-adaptive tools

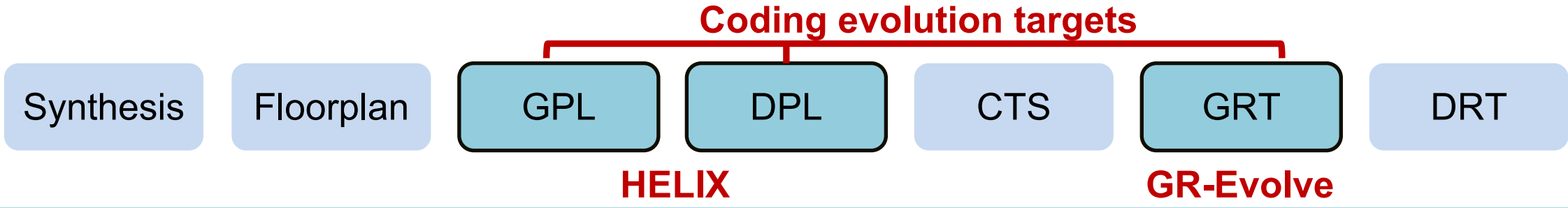
- **Per-design** code evolution; tool **specializes** to the workload
- **HELIX** (UCSD)
  GR-Evolve (ASU)
- **Algorithms change**; one tool per design vs. one tool for all designs

"~9% HPWL gains"

Outline

- **The agentic era in EDA**
- **Case studies in OpenROAD**
- **Emerging challenges for agentic open-source EDA**

Evolving placement and routing in OpenROAD



Attributes	HELIX (placement) – UCSD	GR-Evolve (routing) – ASU
Target	Global + detailed placement (**gpl, dpl**)	Global routing (**grt**)
Method	(μ+λ) evolutionary search with literature-guided operators	Stateless agentic loop with persistent QoR history
Knowledge base	97 papers + documentation + pseudocode + parameter list	Distilled router summaries + source
Key results	**9% HPWL and 23% RWL** (max) 4.5% / 5.6% geomean	**8.7% post-DR WL** (max)

Repository-grounded coding agents for OpenROAD

Repo graph + **documentation**

Literature-grounded planning

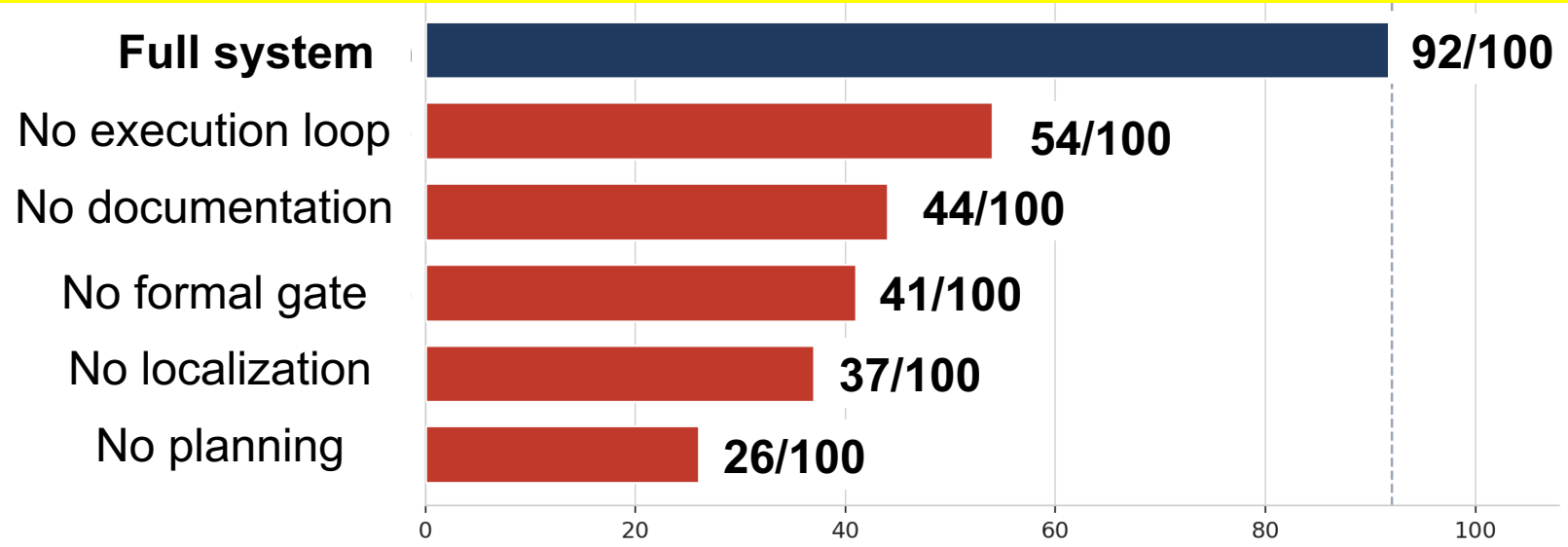
Plan **localization**
(map plan to edit surfaces)

Autonomous execution loop
(apply diffs, build, run flow, measure QoR)

Formal diff **checking**
(Lean; Aristotle; AXLE)

Target	Module(s)	Best Δ	Diff scope
Routed wirelength	dpl	**-5.4%**	11 files, ~1k LoC
Effective clock period	gpl, rsz	**-10.0%**	11 files, ~330 LoC
Total power	rsz	**-19.4%**	9 files, ~1k Loc

Removing any single stage dramatically reduces valid diff-rate!

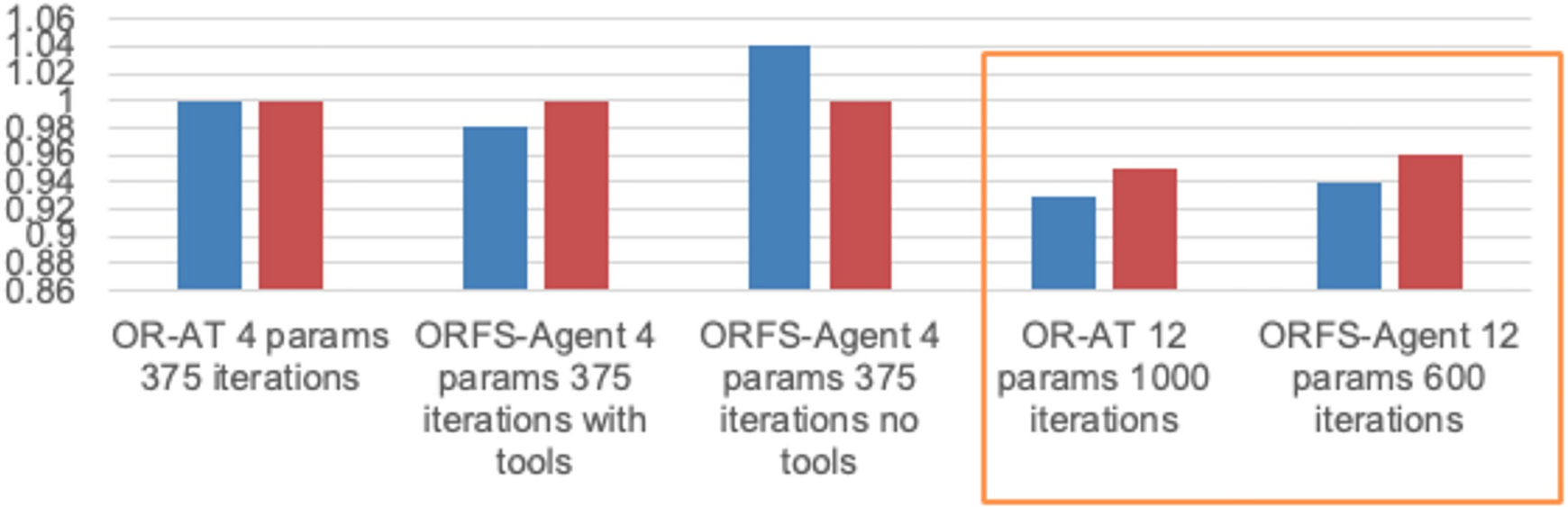


https://arxiv.org/abs/2601.06268

ORFS-agent – LLM-driven flow autotuning

https://arxiv.org/abs/2506.08332

- **Design flow tuning is high-dimensional**
  - 1000s (actually 10000s) of knobs in RTL-to-GDS
- **Plain Bayesian Optimization struggles** – weak domain priors; poor high-D scaling
- **Key idea: LLM-driven flow autotuner → ORFS-agent**
  - Read logs; reason in context; interpret bottlenecks; propose structured parameter updates; iterate with tool feedback



Baseline: OR-AT (4 vars, 375 iters) ≡ 1.0

ORFS-agent uses ~40% fewer iterations, iso-QOR

Also: ≈ 13% gains in WL or ECP (single-objective) (details: MLCAD25 paper)

Comparison of ORFS-agent and OR-AutoTuner w.r.t. **wirelength** and **ECP**

Normalization: Results with OR-AT4 params and 375 iterations set as 1.0

Outline

- The agentic era in EDA
- Case studies in OpenROAD
- Emerging challenges for agentic open-source EDA

Challenge 1: validation & evaluation

- **Agents are non-deterministic; flows are slow and expensive**
  - Reproducibility & traceability – same prompt, different code
  - Cost: minutes to hours per RTL-to-GDS candidate
  - Improvements at GR / placement may only show after detailed routing
- **What "correct" means is no longer obvious**
  - DRC-clean is necessary; not sufficient
  - Multi-objective: WL, timing, power, area, runtime
  - Generalization across designs and PDKs
- **What we need**
  - **Standardized testbeds**: ORFS, MacroPlacement, etc.
  - **Hard gates**: DRC, build, regression tests – reject before scoring
  - **Formal contracts where possible**: Lean / SMT / static analysis
  - **Statistical rigor**: seed sweeps, confidence intervals, ablations

Challenge 2: humans + agents; IP & creators' rights

- **Where does the human belong in the loop?**
  - Objective definition, trade-offs, safety -- not API debugging
  - Agents explore; humans judge and are accountable
  - **Risk:** deskilling if agents become opaque oracle "experts"

- **IP and creators' rights are murky** *(in the 'agentic future' …)*
  - **Who owns an LLM-evolved diff?** Model? Prompter? Repo? *(If it reads on a patent claim?)*
  - Permissively-licensed code bases might accept agent diffs *(is usability clouded?)*
  - **Training** with public "scraped" corpora *(creator rights, opt-ins, … ?)*
  - **Attribution and credit** when an agent rediscovers known techniques

- ***What does "authorship" mean for agent-generated EDA code?***

Challenge 3: project & ecosystem governance

- **Agents change the contribution model**
  - PRs at machine speed: **how does review scale?**
  - Bot accounts, auto-generated commits → **what counts as a maintainer signal?**
  - **Risks:** hallucinated, or subtly-wrong code overwhelming reviewers
- **Ecosystem-level questions**
  - **Reproducibility**: pin commits, PDKs, agents, prompts
  - **Saturated benchmarks:** need open, evolving evaluation suites
  - **Compute access:** who can run repo-scale evolution? ← **equity in research**
  - **Co-evolution:** agent capabilities and tool APIs both **moving targets**
- **In an ideal (OpenROAD) future …**

Note: https://github.com/ieee-ceda-datc/OpenROAD-Research

  - **CI-enforced QoR gates**: non-regressing diffs, machine-checkable invariants
  - **Provenance metadata**: model, prompt, evaluation context per diff
  - **Versioned manifests**: OpenROAD commit + ORFS + PDK + agent → **replay any result**

Toward an open vertical environment for EDA agents

Environment scaffolding (The substrate agents act inside)	Agent capabilities (What we want agents to do)	Ecosystem and governance (What OpenROAD can achieve)
***Episode schema* for EDA** (workspace, actions, constraints, QoR)	**Lexicographic, constraint-aware rewards** (hard gates → QoR frontier → cost)	***Open evolving* agent benchmark suite**
Cross-stage state & observability (RTL ↔ verif ↔ synth ↔ PD ↔ signoff causality)	***Cross-tier* orchestration** (flow-level + code-level in one loop)	***Reproducibility* manifests** (pinned commits, PDKs, agent versions, prompts)
Multi-fidelity ladder (lint → sim → synth → PD → signoff audit)	**Trajectory-grounded training + research-grade memory**	**CI-enforced QoR gates + machine-checkable invariants**
Fast learned surrogates as *first-class env* citizens	**Code-level R&D**	**Industrial calibration loop + compute equity**

AuDoPEDA: *Documentation → Planning → Diff → QoR Loop*

Convert codebase to structured knowledge

- Parse OpenROAD via tree-sitter
- Build cross-language property graph
- Generate hierarchical doc cards
- Create machine-readable API/invariant summaries

Planning is typed and validated

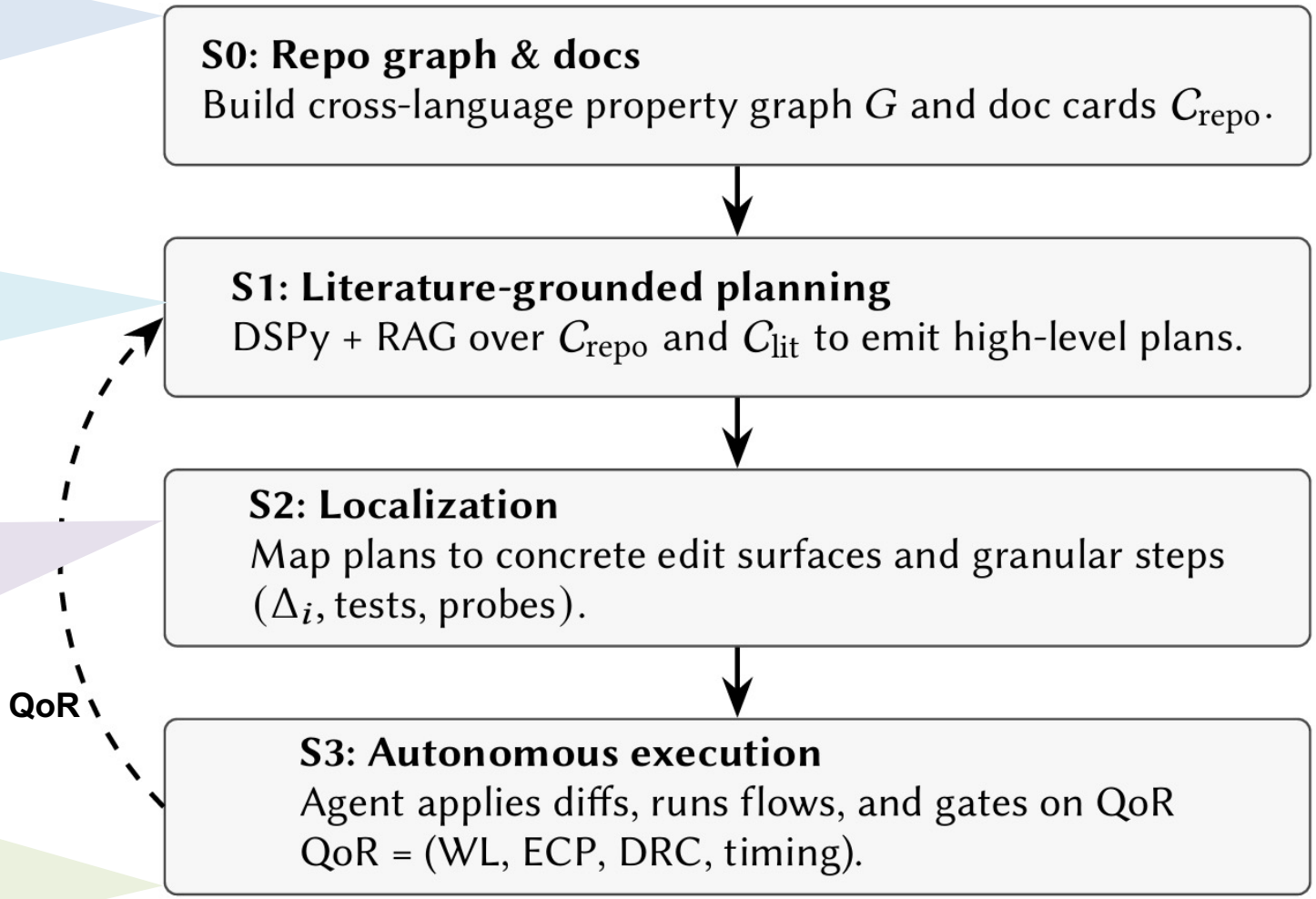
- Retrieve from repo docs, EDA literature corpus
- DSPy-compiled planning program
- Outputs: research hypothesis, proposed interventions, QoR targets

Converts research intent into executable diffs

- Map plan → specific files/functions
- Generate granular diff plan: pre-checks, run config, QoR probes, rollback conditions

Accept only if QoR improves

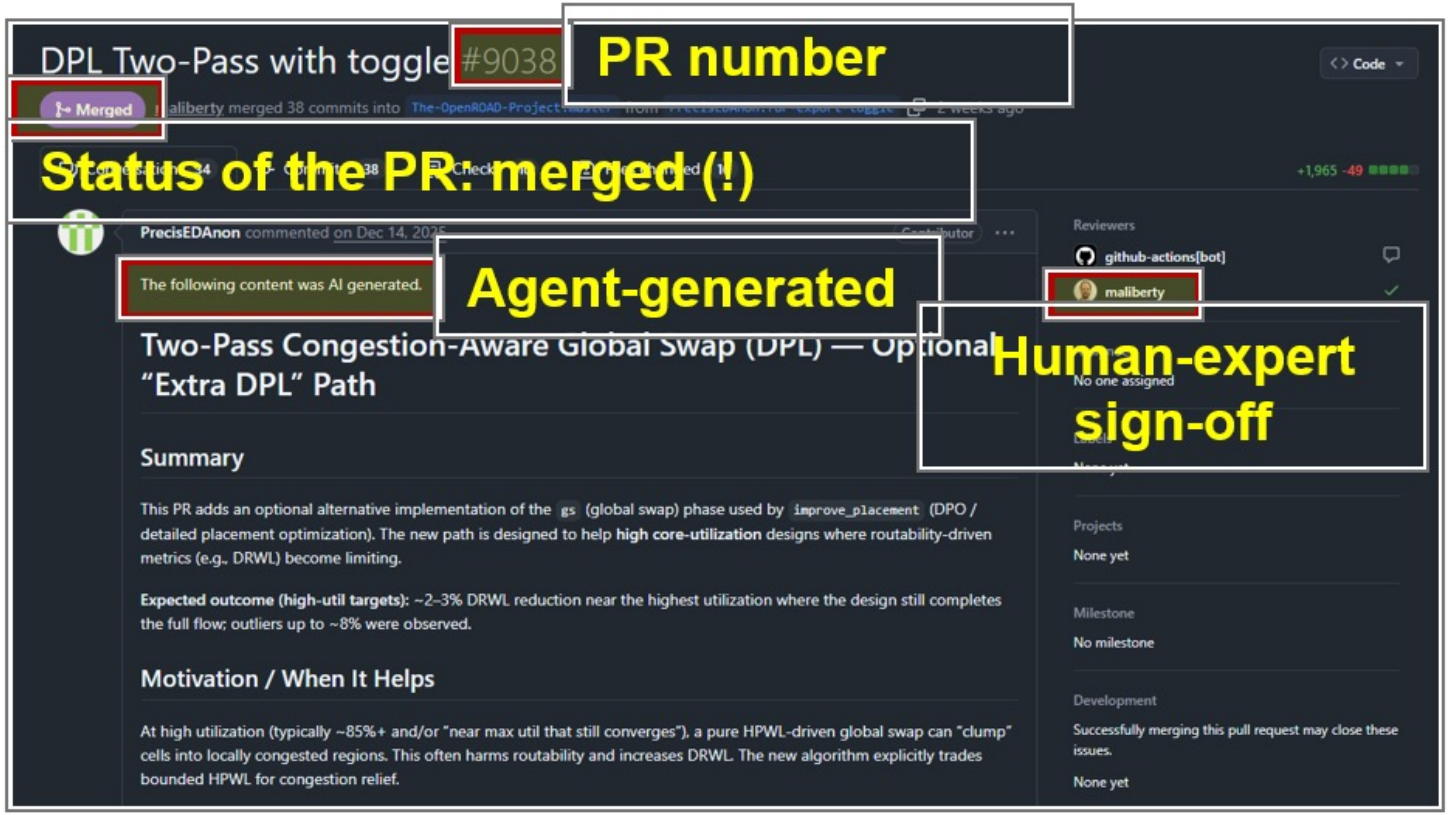
- Codex-class agent
- Apply diffs
- Build
- Run flow
- Extract QoR
- Hill climb under safety gates



A merge-ready OpenROAD PR *(agent-driven)*

What "merge-ready" requires

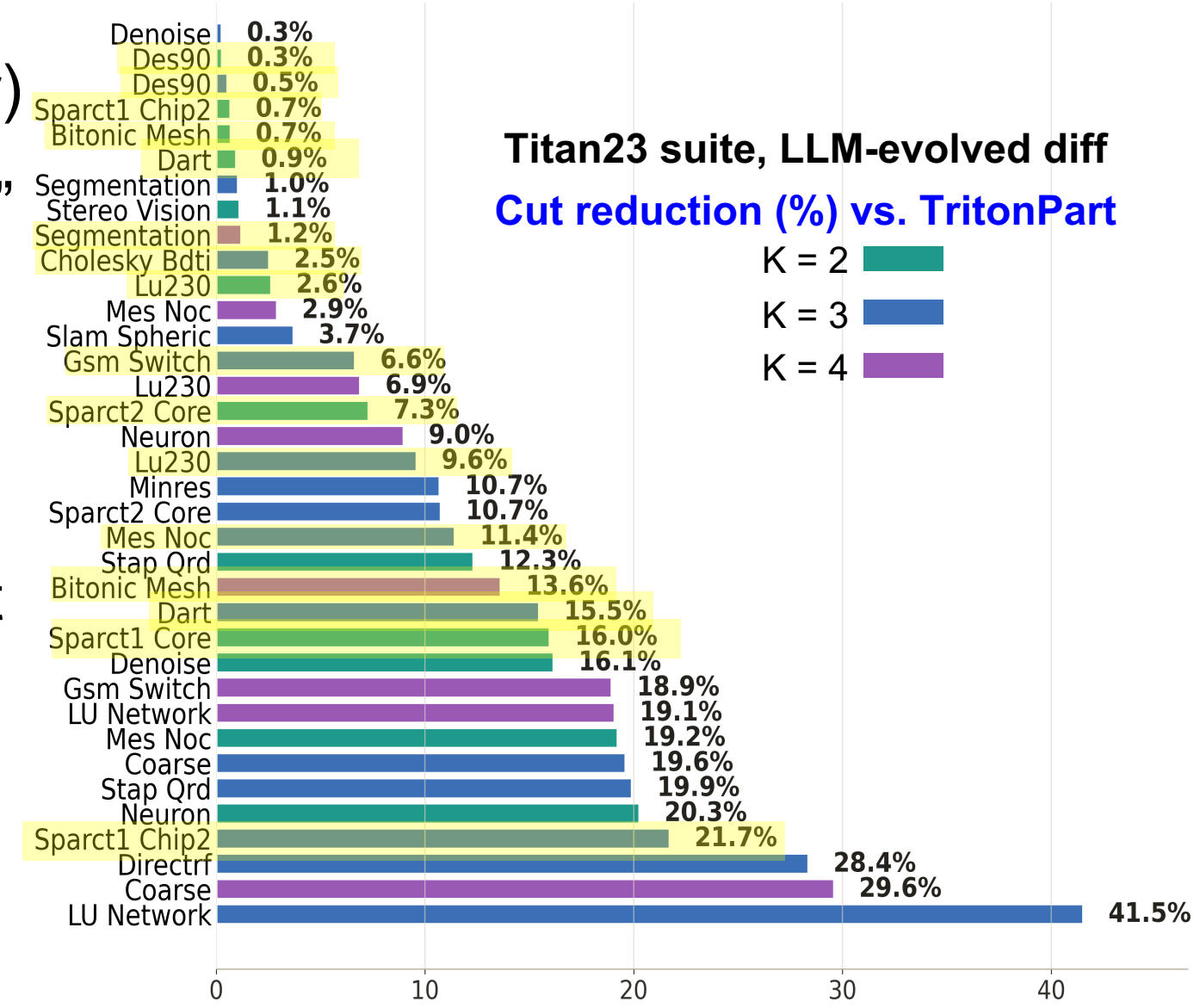
- Passed regression/PPA guardrails
- Iterated on review feedback + merge conflicts
- Auto-generated docs + configs + PR descriptions
- Only manual step: human-expert sign-off
- End-to-end LLM cost: ~$30
- Human-edits: 0 (except sign-off)



https://github.com/The-OpenROAD-Project/OpenROAD/pull/9038

Improving a SOTA Hypergraph Partitioner (OpenROAD/par)

- TritonPart = SOTA hypergraph partitioner (> hMETIS, KaHyPar)
- Has ILP “boundary optimization” step (with CPLEX or OR-Tools)
- **Autonomous AI diff “lowers the variance” of solver step**
- Substantial wins over TritonPart
- Even with the weaker OR-Tools solver, finds **world’s best-ever** solutions!



Links

- GitHubs/Docker Hubs/etc.
- ORFS-Agent: https://github.com/ABKGroup/ORFS-Agent
- EDA corpus: https://github.com/OpenROAD-Assistant/EDA-Corpus
- OpenROAD-Agent: https://github.com/OpenROAD-Assistant/OpenROAD-Agent
- OpenROAD-Assistant: https://github.com/OpenROAD-Assistant/OpenROAD-Assistant.git
- EDA-AI: https://github.com/Thinklab-SJTU/EDA-AI/tree/main
- EvoPlace: https://github.com/yaoxufeng/EvoPlace/tree/master
- ChatEDA: https://github.com/wuhy68/ChatEDA
- AuDoPEDA: https://hub.docker.com/r/abkgroupucsd/audopeda
- HELIX: https://github.com/ABKGroup/HELIX.git

More Links

- MLCAD26 contest: https://github.com/ASU-VDA-Lab/MLCAD26-Contest.git
- GR-Evolve: https://github.com/ASU-VDA-Lab/GR-Evolve
- CHICO-Agent: https://github.com/ASU-VDA-Lab/CHICO-Agent
- OpenROAD: https://github.com/The-OpenROAD-Project/OpenROAD.git
- ORFS: https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.git
- ORFS Docker image: https://hub.docker.com/r/openroad/orfs
- FastPart / hypergraph partitioning artifacts: https://vlsicad.ucsd.edu/hypergraphs/
- MacroPlacement benchmark/infrastructure: https://github.com/TILOS-AI-Institute/MacroPlacement
- CircuitNet: https://github.com/circuitnet/CircuitNet
- CircuitOps: https://github.com/NVlabs/CircuitOps